

PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

// Node structure definition

struct Node {

    int data;

    struct Node* next;

};

// Queue structure definition

struct Queue {

    struct Node* front;

    struct Node* rear;

};

// Function prototypes

struct Node* createNode(int data);

void initializeQueue(struct Queue* queue);

void enqueue(struct Queue* queue, int data);

int dequeue(struct Queue* queue);

void displayQueue(struct Queue* queue);

int main() {

    struct Queue queue;

    int choice, element;

    initializeQueue(&queue);

    while (1) {

        printf("\nQueue Operations Menu:\n");

        printf("1. Enqueue\n");
```

```
printf("2. Dequeue\n");

printf("3. Display\n");

printf("4. Exit\n");

printf("Enter your choice: ");

scanf("%d", &choice);

switch (choice) {

    case 1:

        printf("Enter element to enqueue: ");

        scanf("%d", &element);

        enqueue(&queue, element);

        break;

    case 2:

        element = dequeue(&queue);

        if (element != -1)

            printf("Dequeued element: %d\n", element);

        break;

    case 3:

        displayQueue(&queue);

        break;

    case 4:

        exit(0);

    default:

        printf("Invalid choice! Please enter a valid option.\n");

}

}

return 0;
```

```
}
```

```
// Function to create a new node
```

```
struct Node* createNode(int data) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    if (!newNode) {
```

```
        printf("Memory allocation error\n");
```

```
        exit(1);
```

```
    }
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to initialize the queue
```

```
void initializeQueue(struct Queue* queue) {
```

```
    queue->front = NULL;
```

```
    queue->rear = NULL;
```

```
}
```

```
// Function to enqueue an element to the queue
```

```
void enqueue(struct Queue* queue, int data) {
```

```
    struct Node* newNode = createNode(data);
```

```
    if (queue->rear == NULL) {
```

```
        queue->front = queue->rear = newNode;
```

```
        printf("Element enqueued: %d\n", data);
```

```
        return;
```

```
    }
```

```
    queue->rear->next = newNode;
```

```

    queue->rear = newNode;

    printf("Element enqueued: %d\n", data);
}

// Function to dequeue an element from the queue
int dequeue(struct Queue* queue) {
    if (queue->front == NULL) {
        printf("Error: Queue underflow. Cannot dequeue element.\n");
        return -1;
    }

    struct Node* temp = queue->front;

    int dequeuedElement = temp->data;
    queue->front = queue->front->next;
    if (queue->front == NULL) {
        queue->rear = NULL;
    }

    free(temp);
    return dequeuedElement;
}

// Function to display the queue elements
void displayQueue(struct Queue* queue) {
    if (queue->front == NULL) {
        printf("Queue is empty.\n");
        return;
    }

    printf("Queue elements: ");

    struct Node* temp = queue->front;

```

```
while (temp != NULL) {  
    printf("%d->", temp->data);  
    temp = temp->next;  
}  
printf("NULL\n");  
}
```

OUTPUT:

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 10

Element enqueued: 10

Queue Operations Menu:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter element to enqueue: 20

Element enqueued: 20

Queue Operations Menu:

1. Enqueue
2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter element to enqueue: 30

Element enqueued: 30

Queue Operations Menu:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 3

Queue elements: 10 20 30

Queue Operations Menu:

1. Enqueue

2. Dequeue

3. Display

4. Exit Enter your choice: 2

Dequeued element: 10

Queue Operations Menu:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 3

Queue elements: 20 30

Queue Operations Menu:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 4